# Fast *k*-NN Graph Construction by GPU based NN-Descent

Hui Wang
Xiamen University
Xiamen, China
hwang2019@stu.xmu.edu.cn

Wan-Lei Zhao*
Xiamen University
Xiamen, China
wlzhao@xmu.edu.cn

Xiangxiang Zeng
Hunan University
Changsha, China
xzeng@hnu.edu.cn

Jianye Yang
Hunan University
Changsha, China
jyyang@hnu.edu.cn

## ABSTRACT

NN-Descent is a classic *k*-NN graph construction approach. It is still widely employed in machine learning, computer vision, and information retrieval tasks due to its efficiency and genericness. However, the current design only works well on CPU. In this paper, NN-Descent has been redesigned to adapt to the GPU architecture. A new graph update strategy called selective update is proposed. It reduces the data exchange between GPU cores and GPU global memory significantly, which is the processing bottleneck under GPU computation architecture. This redesign leads to full exploitation of the parallelism of the GPU hardware. In the meantime, the genericness, as well as the simplicity of NN-Descent, are well-preserved. Moreover, a procedure that allows to k-NN graph to be merged efficiently on GPU is proposed. It makes the construction of high-quality *k*-NN graphs for out-of-GPU-memory datasets tractable. Our approach is 100-250× faster than the single-thread NN-Descent and is 2.5-5× faster than the existing GPU-based approaches as we tested on million as well as billion scale datasets.

## CCS CONCEPTS

• **Information systems** → **Information retrieval**; • **Computing methodologies** → **Parallel algorithms**.

## KEYWORDS

k-nearest neighbor graph; GPU; high-dimensional; NN-Descent

**ACM Reference Format:**
Hui Wang, Wan-Lei Zhao, Xiangxiang Zeng, and Jianye Yang. 2021. Fast *k*-NN Graph Construction by GPU based NN-Descent. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM '21), November 1–5, 2021, Virtual Event, QLD, Australia.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3459637.3482344

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*CIKM '21, November 1–5, 2021, Virtual Event, QLD, Australia*
© 2021 Association for Computing Machinery.
ACM ISBN 978-1-4503-8446-9/21/11…$15.00
https://doi.org/10.1145/3459637.3482344

## 1 INTRODUCTION

Given a dataset $S$, the *k*-NN graph is a directed graph structure, in which each node is directed to its top-$k$ nearest neighbors in $S$ under a given distance metric. It is a key data structure in manifold learning [3, 22, 23], machine learning [4] and information retrieval [7, 18], etc. Given the size $n$ and dimension $d$ of a dataset, the time complexity of building a *k*-NN graph exhaustively is $O(d \cdot n^2)$, which is prohibitively expensive for large-scale datasets. Due to the fundamental role of *k*-NN graph in various areas, continuous efforts have been made on the exploration of efficient solutions. Since the time complexity is too high to build an exact *k*-NN graph, most of the works in the literature [6, 7] focus on the construction of approximate *k*-NN graph.

Several efficient approaches [6, 7, 9, 13, 24] have been proposed in recent years. The classic approach NN-Descent [7] was proposed in 2011. It turns out to be a simple but effective solution. According to the paper, only a small portion of the comparisons are required to build a high-quality *k*-NN graph. It is a generic approach in the sense that it is feasible for various distance metrics. Recent approach EFANNA [9] adopts a similar graph refinement strategy as NN-Descent on the rough graphs that are built by the space partitioning strategy. EFANNA demonstrates considerably higher efficiency over NN-Descent. Typically, only a few minutes are required to construct a million-level graph by a single thread. Unfortunately, this scheme is only feasible for $l_p$-norms.

Although both NN-Descent and EFANNA are efficient and easily parallelizable, they are designed for CPU only. As GPU-based computation gets increasingly popular in various fields, more and more researchers explore efficient solutions on GPUs. Thanks to its high performance in floating-point computation, many GPU-based approaches [10–12, 16, 20, 25, 27] have been proposed for approximate nearest neighbor (ANN) search. The most efficient GPU-based NN search is developed based on *k*-NN graph [27]. Unfortunately, the *k*-NN graph construction algorithms in the literature are mostly designed for CPU only. Recent experiments show that building *k*-NN graph exhaustively on the GPU [10, 11, 16] is even much faster than that of NN-Descent on the CPU [7]. Whereas, it is impractical to transplant classic NN-Descent directly to GPU due to the apparent difference in computation architecture. It becomes necessary to explore an efficient solution for *k*-NN graph construction on GPUs. Moreover, another critical issue has been long overlooked in the literature. Up-to-now, there is no solution about how to build a *k*-NN graph when the data is too big to be loaded in the memory.

In this paper, we explore how to construct the large-scale $k$-NN graph on GPU. A GPU-based NN-Descent (*GNND*) is presented. In general, GNND follows the major steps of NN-Descent. However, considerable modifications have been made to adapt the GPU architecture. These modifications include the sampling strategy, distance calculation, and the strategies in graph update. All of these schemes in combination lead to considerably faster speed over the CPU version. Meanwhile, the genericness of the original NN-Descent is well-preserved in GNND. In addition, a simple but effective $k$-NN graph merge approach, that is built upon GNND, is presented. It allows two graphs to be merged efficiently on GPU. More importantly, it makes the $k$-NN graph construction for out-of-GPU-memory dataset tractable via a divide-and-conquer manner.

The remainder of this paper is organized as follows. The related works are reviewed in Section 2. In Section 3, NN-Descent and the architecture of GPU are reviewed as the preliminaries of our presentation in the later sections. Section 4 presents our designs and optimizations of GNND running on a single GPU. In Section 5, our solution for large-scale $k$-NN graph construction is presented. The experimental studies about the effectiveness of the proposed approach are presented in Section 6. Section 7 concludes the paper.

## 2 RELATED WORK

In general, there are two categories of $k$-NN graph construction approaches. The first category follows a divide-and-conquer fashion. The representative approaches are [6, 24, 26]. The second category builds the $k$-NN graph based on iterative refinement on the neighborhoods [7, 14]. NN-Descent [7] is the representative approach of this category.

For divide-and-conquer approaches, the dataset is divided into multiple subsets. The samples in each subset are expected to be relatively close to each other. The division scheme has to be specifically designed. The existing partition approaches include the hierarchical random projections [24], *Recursive Lanczos Bisection* [6] or a series of locality sensitive hash functions [26]. A sub-graph is constructed for each subset in a brute-force way. Finally, these sub-graphs are merged into one $k$-NN graph. Most of these divide-and-conquer approaches are designed for $l_p$-space as it is difficult to define a division scheme for other metric spaces.

For NN-Descent, the $k$-NN graph is constructed by iterative refinement on a random $k$-NN graph. For an object $u$ in the dataset, each iteration of NN-Descent introduces all the neighbors of $u$ to each other. Each neighbor $v$ of $u$ will try to find the neighbor of itself in the neighborhood of $u$. The discovered new neighbors of $v$ will be used to update the neighborhood of $v$. Since most of the operations of NN-Descent are performed in a local of an object, it has a good parallel performance on CPUs. A more detailed review of NN-Descent is presented in the next section. Recently, the hybrid scheme based on both the space partitioning approach and NN-Descent has been proposed in the literature [6]. Although efficient, this hybrid scheme is infeasible for the metrics beyond $l_p$-norms.

Due to the high performance of GPU, several $k$-NN graph construction approaches based on GPUs are also seen in the literature. FAISS-GPU [16] uses a new design for $k$-selection, making brute-force $k$-NN search on GPUs much faster than previous implementations. In the same work, the product quantization (PQ) [15] along with $k$-selection is proposed to construct the $k$-NN graph construction for the large-scale dataset. Whereas extra time costs are introduced for codebook training and quantization. Moreover, high-quality $k$-NN graph cannot be expected due to the quantization loss. GGNN [12] is another representative approach that works on GPU. It constructs $k$-NN graph in hierarchy. GGNN first divides the dataset into several small subsets. The $k$-NN graph for each subset is constructed exhaustively on GPU. A new upper layer is formed by sample objects from each subset on the bottom layer. The sample objects are divided into several subsets, on which a set of sub-graphs are constructed. A hierarchy is built as the sampling and construction continue until there is only one sub-graph on the layer. With the neighborhood relations from the upper layer, a united $k$-NN graph can be built on the lower layer. As this operation propagates to the bottom layer, a complete $k$-NN graph is built for the whole dataset. GGNN uses greedy best first search with backtracking to find the nearest neighbors, a large number of random accesses to the GPU memory becomes inevitable, which impacts its efficiency on GPU considerably.

## 3 PRELIMINARIES

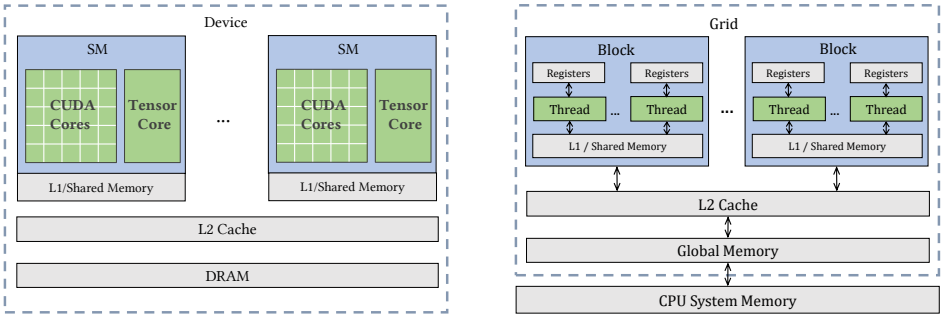
### 3.1 NN-Descent on CPU

NN-Descent [7] is known as a classic $k$-NN graph construction approach. Until now, it is still the most efficient approach that works in generic metric space [18]. The algorithm is built upon the principle of "a neighbor of a neighbor is likely to be a neighbor". In this section, a brief review of it is presented to support the discussions in the later sections.

In general, four steps are involved in NN-Descent. *Step-1.* Initialize a $k$-NN graph with $k$ random neighbors for each object; *Step-2.* Sample a few *new* objects and *old* objects from within $k$-NN list and reverse $k$-NN list of one object. *Step-3.* Perform cross-matching on new-new object pairs as well as the new-old pairs of each object; *Step-4.* Insert the closer neighbors produced from *Step-3* into the $k$-NN list of corresponding objects. The last three steps are repeated until it converges. At the first iteration, all the objects in one $k$-NN list are viewed as *new*. The objects that are newly inserted into a $k$-NN list at *Step-4* are labeled as *new*. Otherwise, the objects in one $k$-NN list are labeled as *old*. According to [7], it is no need to consider all the *new* and *old* objects in the $k$-NN list and reverse $k$-NN list of one object. As a result, sampling in *Step-2* is required. The cross-matching in *Step-3* computes the distances between *new* objects and the distances between *new* and *old* objects. As a result, closer neighbors are produced for the objects joined in the comparison, which are used to update the corresponding $k$-NN lists. Since far neighbors are replaced by the closer neighbors after the insertion each time, the $k$-NN graph evolves to better quality monotonically.

### 3.2 GPU Structures

In this section, we briefly introduce the architecture and programming model of NVIDIA GPU for general-purpose parallel computing. An NVIDIA GPU consists of dozens of streaming multiprocessors (SMs). Each SM contains dozens of processing cores and can execute hundreds of threads concurrently. The simplified hardware architecture of NVIDIA post-Volta GPU is shown in Figure 1(a).




(a) Simplified hardware architecture of NVIDIA post-Volta GPU

(b) Memory hierarchy of CUDA

**Figure 1: Hardware architecture (figure a) and memory hierarchy (figure b) of an NVIDIA post-Volta GPU.**

A function that runs on the GPU is called as a kernel. A grid of thread blocks are launched for one kernel. All threads of one kernel execute the same code. One block is physically corresponding to one SM. A block in an SM is further divided into sub-blocks called warps as the unit of thread scheduling. In CUDA, one warp is comprised of *32* threads. All the threads in a warp follow the Single Instruction, Multiple Thread (SIMT) model. With the support of independent thread scheduling of NVIDIA's post-Volta GPU, diverged threads could run on the same warp, which is much more flexible.

The memory hierarchy is shown in Figure 1(b). Registers are private to each thread. Threads in the same warp could exchange variables in the registers by using warp shuffle functions. Inside each SM, there is a fast on-chip memory. This memory can be used as either L1 cache or shared memory. The latter is visible to the programmers. The threads in one block are able to communicate with each other via this shared memory. Both registers and shared memory are very efficient to access however are of limited capacity. In contrast, global memory (physically DRAM) has much larger capacity, whereas has higher latency, which becomes the bottleneck for memory access intensive tasks.

The advantage of GPU-based computation over CPU lies in its high parallelization. This is the main reason that the exhaustive $k$-NN graph construction on GPU is considerably faster than building $k$-NN graph by NN-Descent on CPU. This motivates us to run NN-Descent on GPU. However, due to the intensive memory access in this algorithm both for distance computation and data insertion, it is non-trivial to transplant the NN-Descent from CPU to GPU. There are several aspects to be considered. In order to make full use of the GPU's floating-point computing capability, the way of memory access should be optimized. It would be more memory-friendly if all the threads in a warp load contiguous data from the memory. Due to the latency mismatch between GPU core and global memory, it is better to reduce the frequency of accessing to the global memory.

In our design, the computation power of GPU as well as the latency mismatch between different hierarchies of GPU memory are carefully considered. Considerable modifications have been made on the original NN-Descent algorithm. Specifically, the modifications include the sampling strategy on the objects to be joined in the next round cross-matching, the distance computation between object pairs, and the insertion strategy for the produced closer neighbors. All of these strategies in combination lead to significantly faster construction speed while maintaining high-quality of the $k$-NN graph.

## 4 APPROXIMATE $K$-NN GRAPH CONSTRUCTION ON GPU

In this section, an efficient $k$-NN graph construction approach on GPU is presented. Basically, we follow the major steps in the classic NN-Descent. The presentation of this section focuses on the major modifications over NN-Descent.

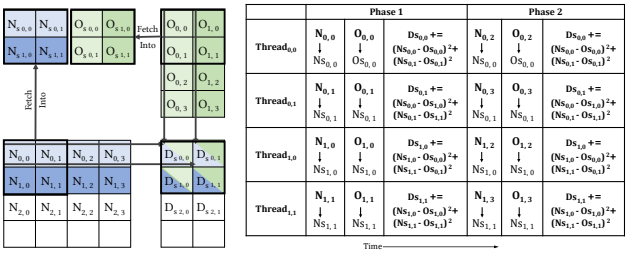
In NN-Descent, distance calculations take up most of the construction time. In contrast, the distance calculation is no longer the processing bottleneck on GPU due to its superior floating-point performance. Because of the latency mismatch between different hierarchies of the memory, the frequent update operations in NN-Descent, which requires intensive memory accesses, become the processing bottleneck. Moreover, the parallel update operations also induce frequent locks and unlocks on each $k$-NN list. As a result, an additional delay is introduced to the whole construction process. The design of our construction algorithm considers how to make full advantage of high GPU floating-point performance and alleviate the delay between the GPU core and memory access in the meantime.

Given a dataset $S$ with $n$ objects, the $k$-NN graph of dataset $S$ is a directed graph with a fixed degree of $k$. For each object $u \in S$, the $k$-NN list of $u$ keeps its top-$k$ nearest neighbors. These $k$-NN lists are sequentially stored in the global memory. Before the iteration starts, all the neighbors in the $k$-NN list of $u$ are marked as *NEW*. After the cross-matching and the update on each $k$-NN list, the neighbors in one $k$-NN list can be divided into two groups. The neighbors that are already in the list in the last iteration are called old neighbors. They are marked as *OLD*. The neighbors that are newly inserted in the current iteration are called new neighbors. They are marked as *NEW*. In the following, $u$'s neighbors marked with *NEW* and *OLD* are called $u$'s *NEW* neighbor and *OLD* neighbor, respectively. The sampled *NEW* and *OLD* neighbors are called *NEW* sample and *OLD* sample, respectively.

**Algorithm 1:** ConstructKNNGraph

**Input:** reference set $S$, number of NN $k$, sample number $p$, the maximum number of iterations $MaxIter$;
**Output:** $k$-NN graph $G$

**parallel for** $s \in S$ **do**
  $G[s] \leftarrow p$ random objects from $S$ ;
  Sort the objects in $G[s]$ in ascending order ;
  Mark the objects in $G[s]$ as $NEW$ ;
**end**
$t \leftarrow$ MaxIter;
**while** $t > 0$ **do**
  $G_{old}, G_{new} \leftarrow$ ParallelSample($S, G, p$) ;
  **parallel for** $s \in S$ **do**
    $S_{new} \leftarrow NEW$ samples in $G_{new}$[s] ;
    $D \leftarrow$ CalculateDistances($S_{new}$) ;
    **parallel for** $u \in S_{new}$ **do**
      $D_u \leftarrow$ distances from other $NEW$ samples to $u$ in $D$;
      $V \leftarrow$ other $NEW$ samples ;
      $(v, d) \leftarrow$ GetNearestObject($u, V, D_u$) ;
      InsertIntoKNNList($G[u], v, d$) ;
    **end**
    $S_{old} \leftarrow OLD$ samples in $G_{old}$[s] ;
    $D \leftarrow$ CalculateDistances($S_{new}, S_{old}$) ;
    **parallel for** $u \in S_{new}$ **do**
      $D_u \leftarrow$ distances from $OLD$ samples to $u$ in $D$ ;
      $V \leftarrow OLD$ samples ;
      $(v, d) \leftarrow$ GetNearestObject($u, V, D_u$) ;
      InsertIntoKNNList($G[u], v, d$) ;
    **end**
    **parallel for** $u \in S_{old}$ **do**
      $D_u \leftarrow$ distances from $NEW$ samples to $u$ in $D$;
      $V \leftarrow NEW$ samples ;
      $(v, d) \leftarrow$ GetNearestObject($u, V, D_u$) ;
      InsertIntoKNNList($G[u], v, d$) ;
    **end**
    Mark all sampled neighbors as $OLD$ ;
  **end**
  $t \leftarrow t - 1$;
**end**
**return** $k$-NN graph $G$ ;




(a) Distance calculation on tiled (b) Two phases distance calculation on tiled

**Figure 2: The illustration of execution phases of tiled distance calculation on a $2 \times 2$ thread block. The subscript $s$ of the variables in the figures indicates that they are defined in the shared memory. In figure (a), the value of the $j$-th dimension of the $i$-th *NEW* sample as $N_{i,j}$. Similarly, $O_{i,j}$ is the value of the $j$-th dimension of the $i$-th *OLD* sample. In figure (b), the "↓" represents that loading one dimension data from global memory to the shared memory. Data in shared memory are accessible for all the threads in one block.**

## 4.1 Sampling on Close Neighbors

In NN-Descent iteration, a $k$-NN list typically consists of OLD neighbors and NEW neighbors after one round of cross-matching and the afterward graph update. The number of *OLD* neighbors and *NEW* neighbors varies from one list to another. Moreover, the numbers of *OLD* neighbors and *NEW* neighbors in the reverse $k$-NN list are unpredictable, which could vary from a few to a few hundreds. If all of these *OLD* and *NEW* neighbors are selected to join the next round of cross-matching, these samples should be collected into a dynamic array for each $k$-NN list. However, the cost of maintaining $n$ dynamic arrays is prohibitively high given the fact that the dynamic memory allocation is of high latency and the global memory might not be sufficient. To address this issue, only the first $p$ ($p < k$) objects are sampled for either *OLD* neighbors or *NEW* neighbors from one $k$-NN list. This results in two adjacency graphs $G_{new}$ and $G_{old}$ in which the sampled $p$ *NEW* neighbors and $p$ *OLD* neighbors are kept for each object separately.

Given graph $G_{new}$ is constructed, each list in $G_{new}$ is visited to append the reverse neighbors derived from $G_{new}$ itself. Namely, given sample $v$ in $G_{new}[s]$, the list of $G_{new}[v]$ is appended with $s$. The appending is an atomic operation. It will be no longer undertaken as long as the size of $G_{new}[v]$ list reaches the upper bound $2p$. The same operation is carried out on the adjacency graph $G_{old}$. As the procedure is running as multi-thread on GPU, this part of the operation can be regarded as random sampling. Since the number of sampled neighbors is fixed, we can store the results of the sampling in the form of fixed-degree graph (fixed-degree adjacency list). An extra array sized of $n$ is defined to keep the sizes of $G_{new}$ and $G_{old}$ lists since not all the lists' sizes could reach $2p$.

In order to reduce the unnecessary distance calculations, we remove duplicates for each list from the resulting graphs $G_{new}$ and $G_{old}$. We simply assign a warp (32 threads) to each list for sorting, and then remove duplicate neighbors from each list. Although this is not the most efficient way on GPU, this simple way is preferred as the time cost of this operation is negligible.

## 4.2 Cross-matching on Sampled Neighbors

As the *NEW* and *OLD* samples are ready, the next step is to perform cross-matching on the sampled lists. Namely, our task is to calculate the distances between the *NEW* samples and the distances between the *NEW* samples and the *OLD* samples. The results of the calculations are kept in the shared memory. Two continuous blocks in the shared memory are allocated to keep these distances for the

*NEW*-*NEW* and *NEW*-*OLD* pairs respectively. As the number of *NEW* samples and the number of *OLD* samples are no bigger than $2p$, the size of these two blocks is small and in fixed length.

For the *NEW*-*NEW* pairs and *NEW*-*OLD* pairs, two different distance calculation strategies are used in our approach. For the *NEW* samples of an object, we use a thread block to perform the distance calculations. Since the number of the *NEW* samples is usually small, all the vectors are loaded into the shared memory. If the data dimension is too high that cannot be loaded into the shared memory at once, they are divided into multiple sub-vectors. The distance calculation is carried out on the sub-vectors. The resulting distances are further reduced into the overall distance between two vectors. Given samples $u$ and $v$, the distance between them is kept at position $\frac{u(u-1)}{2} + v$ of $D_{new}$ in the shared memory. We assign each thread in the thread block with the distance calculation task between two samples. The samples $u$ and $v$ to be calculated by thread-$t$ are obtained as

$$u = \lceil \sqrt{2t + 2.25} - 0.5 \rceil, \tag{1}$$

$$v = t - \frac{u(u-1)}{2}. \tag{2}$$

Due to the limited number of threads in the thread block (*1024* in CUDA), some threads are called for several rounds. For instance, given there are *512* threads in one block, thread-0 should not only calculate $D_{new}[0]$, but also $D_{new}[512]$ and $D_{new}[1024]$. This can be achieved by adding an offset to $t$ in the above equations.

For the distance calculation between the *NEW* and the *OLD* samples, it is treated as a matrix multiplication problem. A variant of tiled matrix multiplication is adopted to calculate the distances between the *NEW* samples and the *OLD* samples. The major difference from the standard tiled matrix multiplication [17] is that the dot product operation in matrix multiplication is replaced with the specified distance metric, such as $l_2$-norm and $K$-Square.

Figure 2 illustrates how the $l_2$-norm is calculated between three *NEW* samples and two *OLD* samples (all of which are *4*-d vectors). In the illustration, we assume there are only four threads (2×2 tiled) in a GPU block. Moreover, we assume there are only three 2×2 cells in the shared memory. One cell could keep a single floating-point value. These 2×2 cells are accessible by all the threads in this block. At the beginning of one calculation, the dimension values from a pair of *OLD* and *NEW* samples are loaded into these 2×2 cells from global memory. As the cells could not hold all the dimensional values (i.e., *8* values) of two vectors at once, the distance calculation on a sample pair is divided into two phases in this example. At the first phase, the first two dimensions of two vectors are loaded and the $l_2$-norm between the sub-vectors is calculated and aggregated. At the second phase, the $l_2$-norm between the remaining two dimensions is calculated and aggregated. It should be noted that the vectors of these samples are stored separately in the global memory, we do not assemble them into a matrix in advance. For this reason, popular libraries such as cuBLAS are not employed. Figure 2 details the sequence of reading the global memory values into the shared memory.

## 4.3 *k*-NN Graph Update

After the cross-matching on the *NEW*-*NEW* sample pairs and the *NEW*-*OLD* sample pairs, distances that are associated with the corresponding sample pairs are kept in the shared memory. In classic NN-Descent, all these produced neighbor pairs are used to update the $k$-NN graph. However, this is too costly in our GPU implementation since the access to the global memory is much slower than the distance calculation on GPU cores. Additionally, locks on the $k$-NN lists are inevitable when a large number of produced pairs are inserted into the graph simultaneously. This slows down the whole process further. As a result, the graph update becomes the processing bottleneck. In this section, two schemes are proposed to address this issue. Firstly, only a few produced neighbors are used to update the graph. Secondly, the time consumption due to graph update is further reduced by setting multiple spinlocks on different segments of a $k$-NN list, which allows the insertion to be fulfilled in parallel on one list.

**Selective update** Different from classic NN-Descent, only very few produced neighbor pairs for one object are selected to update the graph in our approach. In an object local, for a *NEW* sample $u$, we only use the nearest object of $u$ from the other *NEW* samples and the nearest object of $u$ from the *OLD* samples to update the $k$-NN list of $u$. As for an *OLD* sample, we select the nearest object of it from *NEW* samples to update its $k$-NN list. Algorithm 2 summarizes the process to find the nearest object for $u$. Every *32* objects that have calculated distance from $u$ are handled by a warp. If the number of objects is less than *32*, the remaining part will be filled with $(\infty, \infty)$. A standard shuffle based minimal reduction operation (algorithm 2 *Lines* 9–13) on the *32* objects is performed to find the nearest sample of $u$. The CUDA intrinsic *__shfl_down(var, delta)* will copy a variable (*var*) from a thread in the same warp. The second parameter *delta* is the difference between the ID of the target thread and the ID of the caller thread. The warp-level parallel reduction can make full use of the hardware of GPU, which makes the above operations have excellent performance. After the reduction is executed, the nearest object selected by each warp of one group is stored in the variable of *thread-0* of each warp. In the end, the resulting tuple to return will be updated by the variable of *thread-0*.

**Multiple Spinlocks** Typically, three produced neighbor pairs associated with distances are returned by algorithm 2 for one $k$-NN list. They are ready to be inserted into corresponding $k$-NN lists. Threads access to the $k$-NN list in parallel to find the position where object $v$ is to be inserted. Since $k$ is usually small, the above operation can be performed directly in the registers. The neighbors in the $k$-NN list are stored sequentially in the global memory and the accesses to the global memory will be coalesced. As only very few neighbors are selected to insert, the graph update is much faster than inserting all the produced neighbor pairs.

In order to maintain the data integrity, a spinlock should be imposed on the $k$-NN list during an insertion. This is, however, inefficient on GPU. In order to alleviate the latency caused by the spinlocks, the $k$-NN list is divided into multiple segments. Therefore, the spinlock is imposed on the sub-list level, which allows the parallel insertions on one $k$-NN list. Additionally, locks on the sub-list level also reduce the number of threads and global memory accesses required for each insertion. For instance, $k$-NN list is

**Algorithm 2:** GetNearestObject(·)

**Input:** sample $u$, sample set $V$ in which all the distances of samples have been calculated from $u$, distance calculation results $D_u$;
**Output:** a tuple $(v, d)$ which consists of the nearest object $v$ and the distance from $v$ to $u$.

```
1  (v, d) ← (∞, ∞) ;
2  parallel for every 32 objects V_sub ⊆ V do
3      parallel for thread t in a warp do
4          if V_sub[t] exists then
5              (v_t, d_t) ← (V_sub[t], D_u[v_t]) ;
6          else
7              (v_t, d_t) ← (∞, ∞) ;
8          end
9          i ← 16 ;
10         while i > 0 do
11             (v_t, d_t) ← min((v_t, d_t), __shfl_down((v_t, d_t), i)) ;
                /* (v_a, d_a) < (v_b, d_b) when d_a < d_b */
12             i ← i/2 ;
13         end
14         if t == 0 then
15             (v, d) = atomicMin((v, d), (v_t, d_t)) ;
16         end
17     end
18 end
19 return (v, d) ;
```

divided into $k/32$ segments. Each segment keeps *32* (the size of a warp) neighbors. The object $v$ will be inserted into the $v\%(k/32)$-th segment. By applying the optimization, each spinlock only needs to guard one segment, which allows multiple insertion operations to be performed on a $k$-NN list at the same time, thereby reduces the possible waiting time of each thread. In addition, since the number of samples in each segment is the same as the warp size, we only need one warp for one insertion. As a result, much fewer GPU resources are required. As the iteration is completed, all the segments of one $k$-NN list will be merged into one. The speed-up brought by this *multiple spinlocks* will be shown in the ablation study in the experiment section.

The complete GPU-based NN-Descent is presented in algorithm 1. Three major modifications over classic NN-Descent are presented. Firstly, only fixed number of *NEW* samples and *OLD* samples are selected for cross-matching (*Line 8*). During cross-matching, the distances between *NEW* samples and distances between *OLD* samples and *NEW* samples are calculated in different ways to make full use of GPU computation power (*Lines 11* and *19*). Finally, only the discovered close neighbors are selected to update the graph for speed efficiency (*Lines 15*, *23* and *29*).

It is imaginable that the quality of $k$-NN graph gets better after one round iteration since far neighbors are replaced by close neighbors on the $k$-NN lists. However it is possible that the quality improvement might not be as fast as classic NN-Descent as many neighbors are not selected to insert. To confirm the validity of our approach, we study the variation trend of the overall graph structure. A function $\phi(\cdot)$ is defined to measure the overall graph structure

$$\phi(G) = \sum_{i=1}^{n} \sum_{j=1}^{k} G_{ij}, \tag{3}$$

where $G_{ij}$ returns the distance of the $j$-th neighbor to object $i$. The study is conducted on SIFT1M dataset. $\phi(G)$ is calculated after one round of iteration. The comparison is conducted between NN-Descent and GNND. As shown in Figure 3, the variation trend of GNND largely overlaps with that of NN-Descent. A similar trend is observed on other datasets, which implies it is sufficient to use only the closest neighbors to update the $k$-NN list. The other relative far neighbors are quite likely squeezed out later. As a result, ignoring these neighbors during insertion has very little impact on the convergence speed.

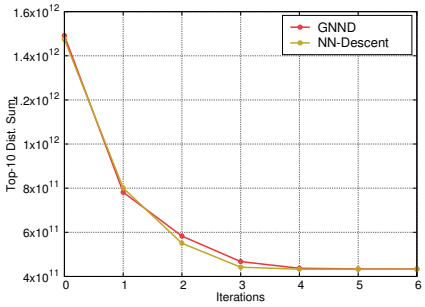



**Figure 3: The variation trend of $\phi(G)$ of GNND in comparison to that of NN-Descent. *k* is fixed to *10* in the experiment.**

## 5 GNND FOR LARGE-SCALE DATASETS

To this end, the proposed algorithm is able to build high-quality $k$-NN graph efficiently. For instance, it only takes a few seconds for million level dataset, which is several hundreds times faster than the classic NN-Descent. However, the problem of $k$-NN graph construction remains challenging given that we have billions of data to be processed. Neither the global memory in GPU nor the system memory could hold such large-scale data at once. Moreover, the large-scale data may not come at once, the $k$-NN graph is required to be constructed incrementally. In this section, a simple but effective $k$-NN graph merge algorithm that builds on GNND is presented. Based on the merge algorithm, both the construction of very large-scale $k$-NN graph and the incremental construction of $k$-NN graph are easily tractable. Additionally, it allows the $k$-NN graph construction to be carried out on multiple GPUs.

### 5.1 *k*-NN Graph Merge

Given two $k$-NN graphs $G_1$ and $G_2$ which are constructed from datasets $S_1$ and $S_2$ respectively[1], the task of $k$-NN graph merge is to build graph $G$ for dataset $S = S_1 \cup S_2$ based on $G_1$ and $G_2$. Intuitively, the graph merge on GPU can be fulfilled by NN search approaches proposed in SONG [27] or GGNN [12]. Namely, graph $G$ can be constructed by querying against graph $G_2$ with objects

[1]Without the loss of generality, we assume $S_1 \bigcap S_2 = \emptyset$.

from $S_1$ and repeat the NN search alternatively between $S_2$ and $G_1$. Although it is feasible, the power of GPUs is not fully capitalized.

In this paper, the $k$-NN graph merge problem is addressed as a graph construction problem on a half-baked graph. Given datasets $S_1$ and $S_2$ with $n$ and $m$ objects respectively, we join their $k$-NN graphs $G_1$ and $G_2$ directly into a graph $G$, which consists of $m+n$ $k$-NN lists. For the first $n$ $k$-NN lists in graph $G$, they actually belong to $G_1$. The last $\frac{k}{2}$ objects of one list from this sub-graph are replaced with $\frac{k}{2}$ random objects from $S_2$. Similarly, for the last $m$ $k$-NN lists in graph $G$, we replace the last $\frac{k}{2}$ objects of each list with $\frac{k}{2}$ random objects in $S_1$. After the initialization, GNND is called to refine graph $G$. Different from performing GNND on a raw set, we only need to perform cross-matching on the pairs from the different sub-graphs as the sub-graphs $G_1$ and $G_2$ are fully baked. Specifically, the last $\frac{k}{2}$ objects of each list are marked as *NEW* samples initially. The distances between *NEW* samples will not be calculated during GNND iteration. As a result, GNND turns out to be much faster than it runs on a raw set. This merge procedure is called GPU-based graph merge (GGM) and summarized in algorithm 3.

**Algorithm 3:** GPU-based graph merge (GGM)

**Data:** dataset $S$, $k$-NN Graph $G_1$ of $S_1$ of size $m$, $k$-NN Graph $G_2$ of $S_2$ of size $n$
**Result:** $k$-NN Graph $G$

**Divide** all lists in $G_1$ into $G_1^u$ and $G_1^v$;
**Divide** all lists in $G_2$ into $G_2^u$ and $G_2^v$;
/* Each list in $G^u$ and $G^v$ owns $\frac{k}{2}$ objects */
**for** *i=1; i ≤ m; i++* **do**
  **Append** $G_1^u[i]$ with $\frac{k}{2}$ random samples from $S_2$;
**end**
**for** *i=1; i ≤ n; i++* **do**
  **Append** $G_2^u[i]$ with $\frac{k}{2}$ random samples from $S_1$;
**end**
$G \leftarrow G_1^u \cup G_2^u$;
Call GNND to refine G;
/* GNND is modified to only calculate the distances between objects in different subsets */
$G \leftarrow$ **Merge and Sort** $G$ with $G_1^v$ and $G_2^v$ ;
**return** $k$-NN Graph $G$

Although GGM is simple, it is a critical procedure that allows the $k$-NN graph to be constructed incrementally. As the new data come in, GNND is called to build a sub-graph on the first hand. Thereafter, GGM is called to join this new sub-graph into the existing $k$-NN graph. Similarly, GGM allows the $k$-NN graph to be built on multiple GPUs simultaneously. Given a large-scale dataset, it is partitioned into a number of subsets. The sub-graphs are constructed by GNND for all the subsets on different GPUs. The sub-graphs are later merged one-by-one with GGM.

In the era of big data, it is possible that the data is too big that the data vectors cannot be held on the memory. In this case, GGM is employed in a different manner to construct the $k$-NN graph.

Firstly, the large-scale dataset is partitioned into multiple shards. Each shard is small enough that is tractable by one GPU. Thereafter, a $k$-NN graph for each shard is built by GNND and saved back to disk. GGM is called to merge every two sub-graphs of two shards. The merged graph is saved back to disk as two sub-graphs. Each $k$-NN list in either sub-graphs retains the top-$k$ neighbors of the corresponding object. Such merge is carried out between sub-graphs pairwisely which ensures any two sub-graphs has been merged once. The resulting sub-graph is a piece of the whole $k$-NN graph, each of which retains the top-$k$ neighbors from the whole dataset.

Because only the shards that are being processed need to be kept in the memory, the framework can handle the out-of-GPU-memory datasets. Obviously, multiple merges can be run on multiple GPUs, which makes the framework easy to benefit from multiple GPUs. Although the speed of reading and writing disk is very slow, because the merge operation takes a relatively long time and runs on GPU, we can read and write the disk while merging graphs on GPU. Therefore, with a reasonable arrangement of disk reading and writing, the time spent on large $k$-NN graph construction will be roughly equivalent to the GPU running time.

**Table 1: Summary on Datasets used for Evaluation**

| Name | $n$ | $d$ | Type |
|---|---|---|---|
| SIFT1M [15] | $1\times10^6$ | 128 | SIFT [19] |
| DEEP1M [1] | $1\times10^6$ | 96 | Deep |
| GIST1M [8] | $1\times10^6$ | 960 | GIST [8] |
| GloVe [21] | $1\times10^6$ | 100 | Text [21] |
| SIFT10M [15] | $1\times10^7$ | 128 | SIFT |
| SIFT100M [15] | $1\times10^8$ | 128 | SIFT |
| SIFT1B [15] | $1\times10^9$ | 128 | SIFT |
| DEEP100M [1] | $1\times10^8$ | 96 | Deep |
| DEEP1B [1] | $1\times10^9$ | 96 | Deep |

## 6 EXPERIMENTS

In this section, the performance of GNND is studied in comparison to the representative approaches in the literature. They are FAISS-IVFFlat [16], FAISS-BF [16], GGNN [12], and classic NN-Descent [7]. NN-Descent is treated as the comparison baseline, which is also the only approach that runs on CPU. FAISS-IVFFlat is a non-exhaustive $k$-NN search approach which uses inverted file as index. FAISS-IVFFlat is highly optimized on GPU. FAISS-BF constructs $k$-NN graphs on the GPU in a brute-force way. Namely, each sample is compared against the rest of the dataset to get its top-$k$ neighbors. GGNN constructs graph index in hierarchy. The bottom layer of the index is a $k$-NN graph for the whole dataset. In addition to FAISS-IVFFlat, FAISS-BF and GGNN, there are other GPU approaches that can be used to construct $k$-NN graph, such as FAISS-IVFPQ [16] and Sweet KNN [5]. Unfortunately, they are very slow on million-scale datasets. For instance, the time cost of building indexing alone in FAISS-IVFPQ is already significantly higher than the cost of building a $k$-NN graph by FAISS-BF. The performance of Sweet KNN on such scale dataset is also much poorer than FAISS-BF. Therefore, Sweet KNN is not considered in our comparison. FAISS-IVFPQ is only considered in billion-scale graph construction experiments.

The experiments are conducted on nine popular ANN benchmark datasets. The scale of these data sets ranges from millions to billions.

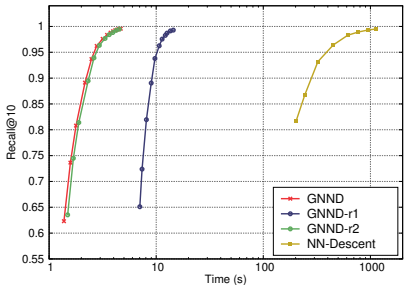
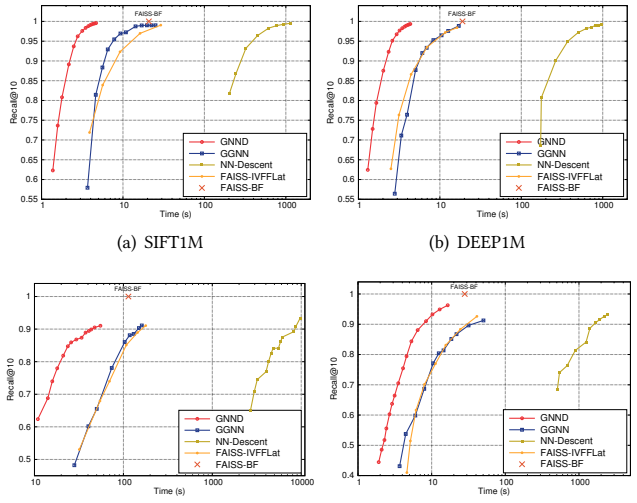



**Figure 4: Ablation study over two proposed schemes, namely *selective update* and *multiple spinlocks*. GNND-r1 is a basic GPU implementation without the proposed schemes. GNND-r2 is integrated with *selective update*. GNND is integrated with both schemes.**




(a) SIFT1M (b) DEEP1M

(c) GIST1M (d) GloVe1M

**Figure 5: The $k$-NN graph construction performance on four million-scale datasets. In this experiment, parameters $k$ and $p$ are varied to produce the curves.**

The brief information of the datasets is summarized in Table 1. All the experiments are carried out on a machine with one Intel Xeon Processor W-2123 (3.60Ghz) and 32 GB of memory. The GPU we use is an NVIDIA GeForce RTX 3090.

## 6.1 Evaluation Protocol

The top-*10* recall rate (*Recall@10*) is used to evaluate the quality of the constructed $k$-NN graphs. Given function $R(i,k)$ returns the number of truth-positive neighbors at top-$k$ NN list of object $i$, the recall at top-$k$ on the whole graph is given as

$$Recall@k = \frac{\sum_{i=1}^{n} R(i,k)}{n \times k}. \quad (4)$$

*Recall@1* is not considered simply because most of the approaches (except for FAISS-IVFPQ) could achieve high score in terms of the top-*1* recall. In our evaluation, the parameters $k$ and $p$ of our approach are varied in a range to produce the *Recall@10*-time curve. Similarly, for other approaches in the comparison, the critical parameters are tuned to produce the *Recall@10*-time curve. For datasets below *100* million, the ground-truth is built for the whole dataset by exhaustive NN search. For datasets of *100* million and *1* billion levels, the ground-truth is only built for *1000* samples since it is intractable to build the ground-truth for the whole set.

## 6.2 Ablation Study

In the first experiment, the speed-up that is achieved by *selectively update* and *multiple spinlock* in GNND is studied. As the strategies in sampling (Section 4.1) and the distance calculations on GPU (Section 4.2) are indispensable parts of our approach, they are integrated with GNND by default. Three runs are carried out in this ablation study. For GNND-r1, all the produced neighbor pairs by cross-matching are used to update the corresponding $k$-NN list. Different from insertion performed on CPU, all the calculated distances to a sample $u$ will be sorted by *Batcher's bitonic sorting network* [2]. These sorted distances and their corresponding objects are then merged with the $k$-NN list of $u$. In GNND-r2, the proposed *selectively update* is integrated. algorithm 2 is called to select the nearest neighbors for insertion. In the third run, namely the full version of GNND, the multiple spinlocks are further set on the $k$-NN lists that allow the insertion to be undertaken in parallel. Please be noted that all of our runs are performed on a single GPU.

In Figure 4, we show the performance from GNND, NN-Descent, GNND-r1, and GNND-r2. As shown in the figure, more than three times speed-up is observed when *selectively update* is integrated into our approach. Compared to GNND-r2, another *5-8%* speed-up is achieved in GNND as the multiple spinlocks allow the parallel insertion in one $k$-NN list. We also learn that the basic GPU implementation of NN-Descent is more than ten times faster than the classic NN-Descent on CPU. This wide performance gap largely dues to the high parallelization of distance computation on GPU.

## 6.3 Million Scale $k$-NN Graph Construction

In this section, the performance of GNND is compared to NN-Descent [7], FAISS-IVFFlat [16], FAISS-BF [16] and GGNN [12] on four million level datasets SIFT1M, DEEP1M, GIST1M, and GloVe1M. For FAISS-IVFFlat, the number of centroids are varied from 2048 to 16384 and the searching parameter *nprobe* is tuned to achieve various graph quality. For GNND, the parameter $k$ is tuned to achieve a good trade-off between graph quality and efficiency. For GGNN, the number of NN $k$ is fixed to *24*, which is in line with the experiments in [12] and we find it is efficient on all datasets. The other two parameters slack variable $\tau$ and refinement iterations $t$ are tuned to produce $k$-NN graphs of different quality. For DEEP1M and SIFT1M, the parameter $k$ is fixed to *64* while parameter $p$ is varied from *10* to *32*. For GIST1M and GloVe1M, $k$ is varied from *64* to *128* while $p$ is varied from *10* to *42*. The results are shown in Figure 5.

Figure 5 shows the experimental results. For SIFT1M, GNND is able to construct a $k$-NN graph with *0.99* recall@*10* in less than *4* seconds, which is *240* times faster than the single-thread NN-Descent. The higher quality of the $k$-NN graph, the wider is the speed gap. FAISS-BF takes *21* seconds to construct an exact $k$-NN

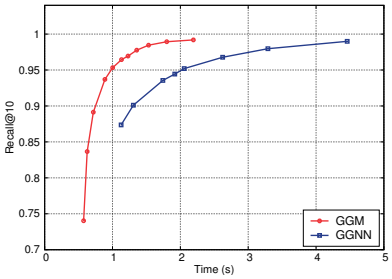



**Figure 6: The performance of merging two $k$-NN graphs. The sub-graphs are constructed by GNND. GGM and GGNN are called to merge the two sub-graphs. The time costs of building the sub-graphs by GNND are not counted in the figure.**

graph for SIFT1M. However, this exhaustive approach is unscalable to datasets with larger scale. For instance, it takes more than *30* minutes to construct an exact $k$-NN graph for SIFT10M. While it only takes around *60* seconds for GNND to reach similar quality. Compared to FAISS-IVFFlat and GGNN, GNND is *2.5* to *5* times faster on the same quality level. This observation is consistent across all the datasets.

## 6.4 Billion Scale $k$-NN Graph Construction

In this section, the performance of GNND is further verified on billion level dataset. Before we show the performance on the billion-scale dataset. The performance of GGM is demonstrated, as GGM is the critical procedure in the whole construction pipeline. In our approach, the billion level graph is constructed by pair-wise merging on a group of million level sub-graphs via GGM. In the first experiment, SIFT1M is divided into two 500K datasets. GNND is called to build the $k$-NN graph for each. Thereafter, GGM is called to merge these two sub-graphs. Alternatively, the two sub-graphs are merged by GGNN [12] for comparison. The results from GNND and GGNN are shown in Figure 6. It is clear to see GGM is consistently better than GGNN by *5-10%* in terms of Recall@10. Compared to GGNN, GGM makes the full use of the constructed sub-graphs. GGNN is unable to merge two $k$-NN graphs directly. Instead, $k$-NN search is conducted with samples from one sub-graph against another sub-graph. As a result, only the neighborhood relations of one sub-graph is used during the search.

With the support of GGM, we further show the performance of our approach in building $k$-NN graph for billion-scale datasets. On this scale, it is no longer tractable for the approaches such as GGNN, NN-Descent or FAISS-BF. They either require the whole dataset to be held in memory (e.g., NN-Descent and GGNN) or are too expensive to be undertaken (e.g., FAISS-BF). For this reason, only FAISS-IVFPQ is considered in our comparison. For our approach, the dataset is partitioned into several hundreds of shards of equal size. Such that we ensure that each shard of the subset is solvable by GNND with a single GPU. After the sub-graphs are constructed by GNND, pairwise merge between the sub-graphs are undertaken as we described in Section 5. For FAISS-IVFPQ, a coarse quantizer with $2^{16}$ centroids is trained in advance. Each vector is encoded into 32 bytes by the quantizer. The distance calculation is conducted between these encoded vectors. Due to the low memory consumption, the large-scale dataset could be loaded into memory to construct the $k$-NN graph.

**Table 2: Performance of $k$-NN graph Constr. on Billion-scale**

| | GNND | | FAISS-IVFPQ | |
|---|---|---|---|---|
| Dataset | Time | *Recall@10* | Time | *Recall@10* |
| SIFT100M | 2,583s | 0.764 | 2,739s | 0.702 |
| SIFT100M | 3,033s | 0.966 | 4,469s | 0.730 |
| DEEP100M | 2,364s | 0.767 | 2,331s | 0.705 |
| DEEP100M | 2,888s | 0.956 | 4,262s | 0.770 |
| SIFT1B | 77h | 0.955 | - | - |
| DEEP1B | 76h | 0.951 | - | - |

The results from GNND and FAISS-IVFPQ are shown in Table 2. Since FAISS-IVFPQ cannot construct $k$-NN graph for a billion-scale dataset on a single GPU due to the limited memory capacity of a single GPU, only the results from GNND are presented for SIFT1B and DEEP1B datasets. As shown in the table, the graph quality from FAISS-IVFPQ is considerably lower than our approach. This is mainly because the distance calculations are performed on the compressed data vectors. One could not expect high graph quality even the exhaustive comparison is conducted on the compressed vectors. Another disadvantage of FAISS-IVFPQ is that it is only feasible for $l_p$-norms.

## 7 CONCLUSION

In this paper, we have presented an efficient $k$-NN graph construction approach that works on GPUs. It largely follows the major steps in classic NN-Descent which only works well on CPU. In order to adapt to the architecture of GPUs, several optimization schemes are proposed. First of all, the dynamic memory allocation is avoided by fixing the number of samples that are joined in the cross-matching. The distance computations have been specifically designed for different types of sample pairs. A large amount of memory accesses are reduced considerably by *selectively update* without any graph quality degradation. The latency of memory access is further minimized by applying *multiple spinlocks* on a $k$-NN list. All these schemes in combination make full use of the parallelism of the GPU hardware.

In addition, an efficient algorithm is presented to merge two $k$-NN graphs into one. Based on the merge algorithm, it becomes possible to construct $k$-NN graph on multiple GPUs. More importantly, the construction of high-quality $k$-NN graphs has been scaled from million level up-to billion level. The empirical study shows that our approach is 100-250× faster than single-thread NN-Descent and is 2.5-5× faster than existing GPU approaches. For out-of-GPU-memory datasets, the quality of the $k$-NN graphs constructed by GNND is significantly higher than the quantization-based approach.

## ACKNOWLEDGMENTS

This work is supported by National Natural Science Foundation of China under grants 61572408 and 61972326, and the grants of Xiamen University 20720180074.

## REFERENCES


[1] Artem Babenko and Victor Lempitsky. 2016. Efficient indexing of billion-scale datasets of deep descriptors. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2055–2063.

[2] Kenneth E Batcher. 1968. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*. 307–314.

[3] Mikhail Belkin and Partha Niyogi. 2001. Laplacian eigenmaps and spectral techniques for embedding and clustering.. In *Nips*, Vol. 14. 585–591.

[4] Oren Boiman, Eli Shechtman, and Michal Irani. 2008. In defense of nearest-neighbor based image classification. In *2008 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 1–8.

[5] Guoyang Chen, Yufei Ding, and Xipeng Shen. 2017. Sweet knn: An efficient knn on gpu through reconciliation between redundancy removal and regularity. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. IEEE, 621–632.

[6] Jie Chen, Haw-ren Fang, and Yousef Saad. 2009. Fast Approximate kNN Graph Construction for High Dimensional Data via Recursive Lanczos Bisection. *Journal of Machine Learning Research* 10, 9 (2009).

[7] Wei Dong, Charikar Moses, and Kai Li. 2011. Efficient K-nearest Neighbor Graph Construction for Generic Similarity Measures. In *Proceedings of the 20th International Conference on World Wide Web (WWW'11)*. ACM, New York, NY, USA, 577–586. https://doi.org/10.1145/1963405.1963487

[8] Matthijs Douze, Hervé Jégou, Harsimrat Sandhawalia, Laurent Amsaleg, and Cordelia Schmid. 2009. Evaluation of gist descriptors for web-scale image search. In *Proceedings of the ACM International Conference on Image and Video Retrieval*. 1–8.

[9] Cong Fu and Deng Cai. 2016. EFANNA: An extremely fast approximate nearest neighbor search algorithm based on knn graph. *arXiv preprint arXiv:1609.07228* (2016).

[10] Vincent Garcia, Eric Debreuve, and Michel Barlaud. 2008. Fast k nearest neighbor search using GPU. In *2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*. IEEE, 1–6.

[11] Vincent Garcia, Eric Debreuve, Frank Nielsen, and Michel Barlaud. 2010. K-nearest neighbor search: Fast GPU-based implementations and application to high-dimensional feature matching. In *2010 IEEE International Conference on Image Processing*. IEEE, 3757–3760.

[12] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik Lensch. 2019. GGNN: Graph-based GPU Nearest Neighbor Search. *arXiv preprint arXiv:1912.01059* (2019).

[13] Rachid Guerraoui, Anne-Marie Kermarrec, Olivier Ruas, and François Taïani. 2020. Smaller, faster & lighter knn graph constructions. In *Proceedings of The Web Conference 2020*. 1060–1070.

[14] Michael E Houle, Xiguo Ma, Vincent Oria, and Jichao Sun. 2014. Improving the quality of K-NN graphs for image databases through vector sparsification. In *Proceedings of International Conference on Multimedia Retrieval*. 89–96.

[15] Herve Jegou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

[16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* (2019).

[17] David B Kirk and W Hwu Wen-Mei. 2016. *Programming massively parallel processors: a hands-on approach*. Morgan kaufmann.

[18] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Wenjie Zhang, and Xuemin Lin. 2019. Approximate Nearest Neighbor Search on High Dimensional Data-Experiments, Analysis and Improvement. *IEEE Transactions on Knowledge and Data Engineering* (2019), 1–1. https://doi.org/10.1109/TKDE.2019.2909204

[19] David G Lowe. 2004. Distinctive image features from scale-invariant keypoints. *International journal of computer vision* 60, 2 (2004), 91–110.

[20] Jia Pan and Dinesh Manocha. 2011. Fast GPU-based locality sensitive hashing for k-nearest neighbor computation. In *Proceedings of the 19th ACM SIGSPATIAL international conference on advances in geographic information systems*. 211–220.

[21] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.

[22] Sam T Roweis and Lawrence K Saul. 2000. Nonlinear dimensionality reduction by locally linear embedding. *science* 290, 5500 (2000), 2323–2326.

[23] Joshua B Tenenbaum, Vin De Silva, and John C Langford. 2000. A global geometric framework for nonlinear dimensionality reduction. *science* 290, 5500 (2000), 2319–2323.

[24] Jing Wang, Jingdong Wang, Gang Zeng, Zhuowen Tu, Rui Gan, and Shipeng Li. 2012. Scalable k-nn graph construction for visual descriptors. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 1106–1113.

[25] Patrick Wieschollek, Oliver Wang, Alexander Sorkine-Hornung, and Hendrik Lensch. 2016. Efficient large-scale approximate nearest neighbor search on the gpu. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2027–2035.

[26] Yan-Ming Zhang, Kaizhu Huang, Guanggang Geng, and Cheng-Lin Liu. 2013. Fast kNN graph construction with locality sensitive hashing. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 660–674.

[27] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1033–1044.